

Model-based Unbiased Learning to Rank

Dan Luo
dal417@lehigh.edu
Lehigh University
Bethlehem, PA, USA

Zhiyu Chen*
zhiyuche@amazon.com
Amazon.com, Inc.
Seattle, WA, USA

Lixin Zou
zoulixin15@gmail.com
Baidu Inc.
Beijing, China

Dawei Yin†
yindawei@acm.org
Baidu Inc.
Beijing, China

Qingyao Ai
aiqy@tsinghua.edu.cn
Tsinghua University
Beijing, China

Brian D. Davison
davison@cse.lehigh.edu
Lehigh University
Bethlehem, PA, USA

ABSTRACT

Unbiased Learning to Rank (ULTR), *i.e.*, learning to rank documents with biased user feedback data, is a well-known challenge in information retrieval. Existing methods in unbiased learning to rank typically rely on click modeling or inverse propensity weighting (IPW). Unfortunately, search engines face the issue of a severe long-tail query distribution, which neither click modeling nor IPW handles well. Click modeling usually requires that the same query-document pair appears multiple times for reliable inference, which makes it fall short for tail queries; IPW suffers from high variance since it is highly sensitive to small propensity score values. Therefore, a general debiasing framework that works well under tail queries is sorely needed. To address this problem, we propose a model-based unbiased learning-to-rank framework. Specifically, we develop a general context-aware user simulator to generate pseudo clicks for unobserved ranked lists to train rankers, which addresses the data sparsity problem. In addition, considering the discrepancy between pseudo clicks and actual clicks, we take the observation of a ranked list as the treatment variable and further incorporate inverse propensity weighting with pseudo labels in a doubly robust way. The derived bias and variance indicate that the proposed model-based method is more robust than existing methods. Extensive experiments on benchmark datasets, including simulated datasets and real click logs, demonstrate that the proposed model-based method consistently outperforms state-of-the-art methods in various scenarios. The code is available at <https://github.com/rowedenny/MULTR>.

CCS CONCEPTS

• Information systems → Learning to rank.

KEYWORDS

Unbiased Learning to Rank; Doubly Robust; User Simulator

† Corresponding author.

* The work was done prior to joining Amazon.



This work is licensed under a Creative Commons Attribution International 4.0 License.

ACM Reference Format:

Dan Luo, Lixin Zou, Qingyao Ai, Zhiyu Chen*, Dawei Yin†, and Brian D. Davison. 2023. Model-based Unbiased Learning to Rank. In *Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining (WSDM '23)*, February 27-March 3, 2023, Singapore, Singapore. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3539597.3570395>

1 INTRODUCTION

Search engines serve as one of the most important tools for accessing information online. In modern search engines, learning to rank (LTR) algorithms play a critical role by creating models to accurately order a list of candidate documents based on their relevance to the query. As deep supervised models have been widely applied and been state-of-the-art in many ranking tasks [8, 13, 48, 51, 56, 58], obtaining large-scale and high-quality training data has become a bottleneck for the development of large scale learning-to-rank systems [5, 57]. In practice, implicit feedback that reflects users' information needs [37] provides natural, abundant sustainable training data for ranking optimization without costly time consumption and human annotation. Therefore, LTR with implicit feedback such as clicks has received considerable attention in the IR community.

However, click data is biased since relevance is not the only factor influencing users' clicks. For example, *position bias* occurs because users are more likely to examine documents at higher ranks [11, 24]. Consequently, the highly ranked document may receive more clicks, and the relevant document may be perceived as a negative sample simply by not being examined by users. Furthermore, ranking positions [23], display differences [38, 50], users' first impression [32], *etc.*, also influence the implicit feedback. These biases make the data deviate from reflecting true relevance, and jeopardize the learned ranking model's performance.

To be unaffected by biases [25], there are two groups of methods: (1) **Click modeling methods** explicitly introduce an additional factor as bias, make hypotheses about users' browsing behaviors, and estimate true relevance by optimizing the likelihood of observed user clicks [7, 11, 15, 45]. Click models are straightforward yet effective, and they have made promising progress for various applications, such as CTR prediction in live recommendation [16] and grid view web applications [54]. However, search engines are faced with severe long-tail query distribution, where click models can fall short since multiple observations for the same query may not be available [2]. (2) **Propensity-based methods** treat the click bias as the counterfactual factor [34], and re-weight the click data for a relevance-equivalent training loss through *inverse propensity*

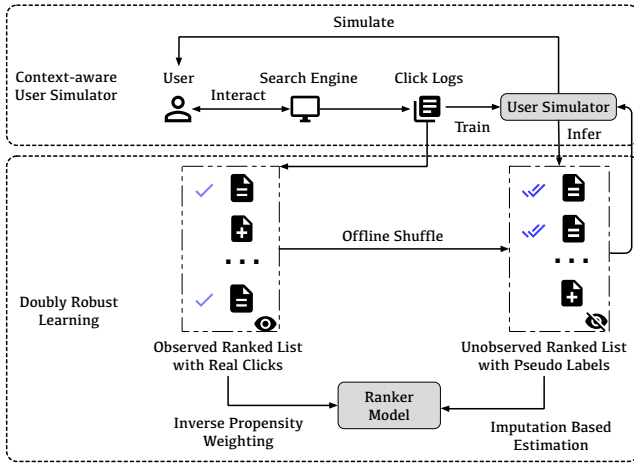


Figure 1: The framework of model-based unbiased learning to rank.

weighting (IPW) [25, 46]. To this end, many approaches [2, 20, 44] are proposed to jointly learn user bias models (i.e., propensity models) with unbiased rankers. However, propensity-based methods generally suffer from high variance [35], which can lead to suboptimal estimation [41]. Therefore, a general debiasing framework with low variance is needed.

To address these challenges, we propose a novel model-based unbiased learning to rank (MULTR) framework, as shown in Figure 1. MULTR consists of a context-aware user simulator for addressing the data sparsity problem and a doubly-robust learning algorithm for reducing the variance and achieving unbiased estimation. Specifically, we first develop a user simulator that directly learns from click data, which generates pseudo clicks for unobserved ranked lists as data imputation and alleviates the sparse training data problem. Remarkably, the user simulator is unbiased since the factor, i.e., the position, influencing the click is being considered as its input, which ensures the consistency between the training of user simulator and data augmentation. However, as the user simulator cannot be as accurate as real users, the discrepancy between pseudo and actual clicks may mislead the ranking models, and unbiasedness cannot be guaranteed. To reduce the variance and ensure the unbiasedness, we resort to doubly robust approaches. In particular, we propose to take the *observation of ranked lists* as the treatment, generate pseudo-click data via *offline result randomization*, and incorporate the inverse propensity weighting with pseudo-click labels in a doubly robust way, where the user simulator essentially functions as the imputation model. The derived bias and variance indicate that the proposed model-based method has low bias and low variance. Thus, it is more robust than existing methods. To demonstrate the effectiveness of the proposed method, we conduct extensive experiments on simulated and real datasets, and demonstrate that the proposed model-based method consistently outperforms state-of-the-art methods.

The contributions of the proposed model-based unbiased learning to rank framework can be summarized as follows:

- We propose model-based unbiased learning to rank (MULTR) – a general debiasing framework with low variance.

- We devise a user simulator, which addresses data sparsity by generating pseudo-click labels for unobserved ranked lists.
- We propose the observation of rank lists as treatment, which addresses the treatment unobservability in unbiased learning to rank, and incorporate inverse propensity weighting with pseudo labels in a doubly robust way.
- We conduct extensive experiments on simulated datasets and real user click logs to demonstrate the superiority of MULTR.

2 RELATED WORK

Unbiased Learning to Rank. To extract unbiased and reliable relevance signals from biased click signals, there are two streams of unbiased learning to rank methodologies. One school depends on click modeling, which makes assumptions about user browsing behaviors. Such methods maximize the likelihood of the observed data, model the examination probability and infer accurate relevance feedback from user clicks [6, 9, 11, 30]. For example, Guo et al. [16] propose a position-bias aware learning framework for CTR prediction that models position-bias in offline training and conducts online inference without biased information. Wang et al. [47] unify the training of the ranker model and the estimation of examination propensity with a graphical model and an EM algorithm. Despite their success, one major drawback of click models is that they usually require that the same query-document pair appears multiple times for reliable inference [29]; thus they may fall short for tail queries. The other school derives from counterfactual learning, which treats bias as a counterfactual factor and debiases user clicks via inverse propensity weighting [25, 46]. Recent efforts jointly model the propensity estimation and unbiased learning to rank. For instance, Ai et al. [2] and Hu et al. [20] present dual learning frameworks for estimating bias and training a ranking model. Vardasbi et al. [43] propose cascade model-based inverse propensity scoring for propensity estimation in the cascade scenario. However, these methods ignore the severe high variance problem in IPW-based methods, particularly for long-tail data. Our paper falls in the same propensity-based framework; however, we address the high variance problem by optimizing the ranking model in a doubly robust way, which has low bias and low variance.

Model-based Methods. Model-based methods aim to construct a predictive user model and ask the question of the counterfactual form “what will the user click if the search result page is presented differently?”, which naturally fits debiasing. Recent work has proposed some methods that improve ranking performance based on user simulators. For example, Dai et al. [12] propose a utility estimator to generate counterfactual data to train the ranking model. Zhang et al. [53] develop a reinforcement learning algorithm to learn ranking policies in the simulation environments. While both methods show promising improvements, the theoretical guarantees with respect to unbiasedness are unclear. Different from such work, we leverage the user simulator as the imputation model in a doubly-robust way, which addresses the high bias issue caused by the discrepancy between user simulator and real clicks. More importantly, our derivation demonstrates the unbiasedness of the proposal. Our ablation study verifies that the doubly robust approach can effectively leverage both real and pseudo clicks, and improve the ranking performance.

Doubly-Robust Methods. Doubly-Robust (DR) methods have been widely applied to position-biased clicks. Saito [36] proposes a DR method for post-click conversions. Guo et al. [17] further develop a more doubly robust estimator to reduce the variance. Kiyohara et al. [26] develop a cascade doubly robust estimator for off-policy evaluation, Yuan et al. [49] introduce a DR estimator for click-through-rate prediction. Zou et al. [55] propose a doubly robust estimator for relevance estimation. A significant difference between current DR estimators and those in ULTR is that existing solutions use corrections based on action propensities, which is similar to generic counterfactual estimation; while ULTR needs examination propensities [31], which unfortunately is unobservable in click logs. In this paper, we address this challenge by taking the observation of rank lists as treatment, and incorporate the inverse propensity weighting with pseudo-click labels in a doubly-robust way.

3 PRELIMINARIES

This section presents the task of unbiased learning to rank and reviews the preliminary methods of click modeling and propensity-based methods with their strengths and weaknesses.

3.1 Task Formulation

Let $\mathcal{D}$ be the universal set of documents, and $\mathcal{Q}$ be the universal set of queries. For a user-issued query $q \in \mathcal{Q}$, we use r_d to denote the relevance annotation over the query q and document $d \in \mathcal{D}$. The goal of learning to rank is to find a mapping function from a query document pair (q, d) to its relevance r_d as $f: \mathcal{Q} \times \mathcal{D} \rightarrow \mathbb{R}$. A corresponding local loss function is usually proposed to learn the best f , given its retrieved ranked list π_q as,

$$\ell_{ideal}(f, q|\pi_q) = \sum_{d \in \mathcal{D}} \Delta(f(q, d), r_d|\pi_q), \quad (1)$$

where Δ is a function that computes the individual loss on each document. $\ell_{ideal}(f, q)$ is the ideal local ranking loss for optimizing the ranking function f with all the documents annotated. Without loss of generality, we simplify the individual loss function $\Delta(f(q, d), r_d|\pi_q)$ as $\Delta(r_d|\pi_q)$.

Typically, the relevance annotation r_d is elicited through expert judgment; thus r_d is considered to be unbiased, but expensive. An alternative approach is to use click data as relevance feedback from users. Suppose there is a click dataset in which the clicks on documents with respect to queries by an initial ranking model are logged. If we conduct learning to rank by replacing the relevance label r_d with click label c_d in Equation 1, then the empirical local ranking loss is derived as follows,

$$\ell_{naive}(f, q|\pi_o) = \sum_{d \in \pi_o, c_d=1} \Delta(c_d|\pi_o), \quad (2)$$

where π_o is the *observed* ranked list, which is the ranked list presented to users, and c_d is a binary variable indicating whether the document d in the ranked list π_o is clicked. However, this naive loss function is biased. For instance, position bias occurs because users are more likely to examine the documents at higher ranks [25]. Consequently, highly ranked documents may receive more clicks, and relevant (but unclicked) documents may be perceived as negative samples because they are unexamined by users. To address

this issue, unbiased learning-to-rank aims to eliminate bias in click data and then train a ranking model with the resulting user clicks.

3.2 Click Modeling

To eliminate biases in click data, one intuitive method is to model bias and relevance as independent factors, and represent the joint effects of bias and relevance with bias-aware click modeling [16, 27]. Here we refer to the general approach as click modeling. Let $\mathbf{b}$ be the bias features, and $\mathbf{x}_d$ be the vector representation of document d , then the bias-aware click predictor is modeled as follows:

$$\hat{P}(c_d = 1, q) = h(\mathbf{b}; \phi) \oplus f(\mathbf{x}_d; \theta), \quad (3)$$

where $\hat{P}(c_d = 1, q)$ is the estimated click probability of document d of query q . h is the bias model parameterized by ϕ that generates a score based on the inputs of bias features. f is the relevance prediction model parameterized by θ that generates a relevance score based on the relevance representations. $\oplus$ is an operation that combines the bias-based score and the relevance score, which could be addition [54], multiplication [16], etc. The general local ranking loss of a bias-aware click predictor is defined as:

$$\begin{aligned} \ell_{CLICK}(f, h, q|\pi_o) &= \sum_{d \in \pi_o, c_d=1} \text{CE}(\hat{P}(c_d = 1, q), c_d) \\ &= \sum_{d \in \pi_o, c_d=1} \text{CE}(h(\mathbf{b}; \phi) \oplus f(\mathbf{x}_d; \theta), c_d), \end{aligned} \quad (4)$$

where c_d is a binary variable indicating whether document d is clicked, and $\text{CE}(\cdot, \cdot)$ the cross-entropy function. When conducting the relevance inference, one can easily eliminate the influence of bias by dropping the scores from $h(\mathbf{b}; \phi)$, i.e., directly ranking with $f(\mathbf{x}_d; \theta)$. Though click models work well with head queries, they could fall short when multiple observations of the same query may not be available [2].

3.3 Propensity-based Methods

Inverse propensity weighting (IPW) is the first unbiased learning to rank algorithm proposed under the propensity-based framework [25, 46]. Let e_d, c_d be the binary variables that represent whether document d is examined and clicked by a user, based on the *Examination Hypothesis* [33] that a user would only click a document when it is observed by the user and considered relevant to the user's need, we have

$$c_d = 1 \iff (e_d = 1 \text{ and } r_d = 1). \quad (5)$$

Then, IPW instantiates the local ranking loss as

$$\ell_{IPW}(f, q|\pi_o) = \sum_{d \in \pi_o, c_d=1} \frac{\Delta(c_d|\pi_o)}{\hat{P}(e_d = 1)}, \quad (6)$$

where $\hat{P}(e_d = 1)$ is the estimated probability that document d is examined in the query session. A nice property is that only clicked documents $c_d = 1$ contribute to the estimation in Eq. 6.

Bias and Variance Analysis. We analyze the bias and variance of the inverse propensity weighting approach.

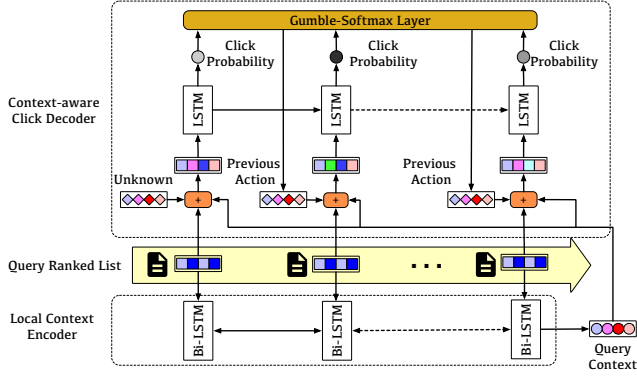


Figure 2: Framework of the Context-aware User Simulator.

THEOREM 3.1. *The bias of the IPW estimators is*

$$\text{Bias}[\ell_{IPW}(f, q|\pi_o)] = \left| \sum_{d \in \pi_o, r_d=1} \frac{P(e_d=1) - \hat{P}(e_d=1)}{\hat{P}(e_d=1)} \Delta(c_d|\pi_o) \right| \quad (7)$$

As shown in Theorem 3.1, when the estimated examination probability is accurate, i.e., $P(e_{d_i}=1) = \hat{P}(e_{d_i}=1)$, then $\text{Bias}[\ell_{IPW}] = 0$, indicating accurate estimation can be achieved with rich observation data. Then, we derive the variance of the IPW estimators.

THEOREM 3.2. *The variance of the IPW estimator is*

$$\mathbb{V}_O[\ell_{IPW}(f, q|\pi_o)] = \sum_{d \in \pi_o, c_d=1} \frac{P(e_d=1)(1-P(e_d=1))}{\hat{P}(e_d=1)^2} \Delta(c_d|\pi_o)^2. \quad (8)$$

Theorem 3.2 illustrates that the variance of IPW estimators depends on the estimated propensity. When $\hat{P}(e_d=1)$ is small, it may lead to high variance. Especially for long-tail queries with rare observation data, the high variance of IPW becomes a problem that directly influences the effectiveness of the ranker.

4 MODEL-BASED UNBIASED LEARNING TO RANK

In this section, we first introduce the context-aware user simulator, which generates pseudo-click labels for unobserved ranked lists as data imputation and addresses the data sparsity. Afterwards, we propose a doubly robust estimator, which takes the observation of ranked lists as the treatment, and further incorporates inverse propensity weighting with pseudo labels from the user simulator above in a doubly robust way. Lastly, we derive the bias and variance of the proposed method, and demonstrate that our proposed method is more robust than existing methods.

4.1 Context-aware User Simulator

The user simulator consists of two important components: a local context encoder, which captures different feature distributions from different queries, and a context-aware click decoder, which produces the click probability of each document sequentially. The framework of our user simulator is shown in Fig. 2.

Local Context Encoder. To capture the characteristics of queries, i.e., different queries may have different distributions in the feature

space [1], we first design a local context encoder for each query and use it to refine the query-specific features, as shown at the bottom block in Figure 2. Formally, given the top N documents $\{d_i\}_{i=1}^N$ in a ranked list π_q from top to bottom, and their corresponding feature vectors $\{x_i\}_{i=1}^N$, we use a bi-directional long short-term memory [19] network to obtain a contextual representation of each document with respect to the entire ranked list:

$$\begin{aligned} \mathbf{h}_i &= [\overrightarrow{\text{LSTM}}(\mathbf{x}_i, i); \overleftarrow{\text{LSTM}}(\mathbf{x}_i, i)] \\ \mathbf{h}_{\pi_q} &= \mathbf{W}^T \mathbf{h}_N, \end{aligned} \quad (9)$$

where $[\cdot; \cdot]$ is concatenation, and $\overrightarrow{\text{LSTM}}(\mathbf{x}_i, i)$ processes the document x from top to bottom and returns the LSTM hidden state at position i (and vice versa for the backward direction $\overleftarrow{\text{LSTM}}$). We then produce the local context representation $\mathbf{h}_{\pi_q}$ by a linear transformation of the last hidden state $\mathbf{h}_N$.

Context-aware Click Decoder. To characterize different influences from the previous actions, we encode actions with an embedding matrix $\mathbf{A}$. In our settings, there are three types of actions, i.e., click, skip, and unknown, where unknown is designed for the initial step. We use $\mathbf{h}_{a_t}$ to denote the action embedding corresponding to action a_t at step t . The context-aware click decoder is formulated as:

$$\begin{aligned} \mathbf{z}_\pi &= \mathbf{W}_1^T \mathbf{h}_{\pi_q} + \mathbf{b}_1 \\ \mathbf{z}_{x_i} &= \mathbf{W}_2^T \mathbf{x}_i + \mathbf{b}_2 \\ \mathbf{z}_{a_t} &= \mathbf{W}_3^T \mathbf{h}_{a_t} + \mathbf{b}_3 \\ \mathbf{z}_t &= [\mathbf{z}_\pi; \mathbf{z}_{x_i}; \mathbf{z}_{a_t}] \\ \mathbf{h}_t &= \overrightarrow{\text{LSTM}}(\mathbf{z}_t, t), \end{aligned} \quad (10)$$

where $\mathbf{W}_1, \mathbf{W}_2, \mathbf{W}_3, \mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_3$ are the trainable parameters used to transform the query context vector, the document vector and the previous action embedding, respectively.

At each step t , the hidden state $\mathbf{h}_t$ is projected into a conditional click probability score through the sigmoid function:

$$p_t = \text{sigmoid}(\mathbf{W}_4^T \mathbf{h}_t + b_4), \quad (11)$$

where $\mathbf{W}_4, \mathbf{b}_4$ are trainable parameters, and $p_t \in [0, 1]$ represents the probability that a user clicks document d_t .

User Simulator Optimization. After obtaining the click probability score p_t for each document, we train the user simulator by applying the binary cross-entropy:

$$\ell_e(g, q|\pi_o) = \sum_{t=1}^N (-c_t \log p_t - (1 - c_t) \log(1 - p_t)) + \lambda \|\phi\|^2, \quad (12)$$

where g is the whole context-aware user simulator, c_t denotes the click label of document d_t in observed ranked list π_o , ϕ denotes all the parameters of the user simulator g , and λ denotes the L_2 regularization coefficient.

Discussion. It is worth noting that the goal of this paper is not to propose a high-performance user simulator. More sophisticated models, such as utility estimator [12] and UBS4RL [52], can be applied here. We leverage the user simulator as the imputation model to generate pseudo-click labels for an arbitrary ranked list. In the next section, we will leverage the user simulator in a doubly robust way to obtain unbiased ranking models.

4.2 Doubly Robust Learning

User simulators have the benefit of data imputation [18]. However, directly optimizing the ranking model through the user simulator could be highly biased due to the discrepancy between pseudo labels and actual clicks, *i.e.*, the pseudo clicks cannot be as accurate as real clicks. To address this problem, we resort to the doubly robust approach. Unfortunately, the treatment, *i.e.*, document examination, is not directly observable in click data. It is because when a document is not clicked, we cannot determine whether the user chose not to click or the user did not examine it [31]. To overcome this bottleneck, we propose to take the *observation of ranked lists* as the treatment. After that, we incorporate inverse propensity weighting with pseudo-clicked labels in a doubly robust way, where the user simulator essentially functions as the imputation model.

Before we describe our doubly robust estimator for unbiased learning to rank, we introduce two concepts: prediction error and imputation error. We refer to the local ranking loss ℓ_{naive} in Eq 2 with actual clicks as *prediction error*. Then the *imputed error* ℓ_{IMP} , *i.e.*, the estimated value of the prediction error, is defined as:

$$\ell_{IMP}(f, q|\pi_o) = \sum_{d \in \pi_o, \hat{c}_d=1} \Delta(\hat{c}_d|\pi_o), \quad (13)$$

where $\hat{c}_d$ is the imputed click generated from the user simulator for each document d in the observed ranked list π_o under query q .

Let Π_q be the set of document permutations, *i.e.*, all possible ranked lists. We simplify Π_q as Π when there is no ambiguity for the issued query q . We propose our doubly robust (DR) estimator by combining the imputed errors for all ranked lists, and the IPW-based prediction error for observed ranked lists. The loss function of the DR estimator given query q is defined as:

$$\begin{aligned} \ell_{DR}(f, q|\pi_o) &= \frac{1}{|\Pi|} \sum_{\pi \in \Pi} \left\{ \hat{\ell}_{IMP}(f, q|\pi) \right. \\ &\quad \left. + o_\pi \left[\ell_{IPW}(f, q|\pi) - \hat{\ell}_{IPW}(f, q|\pi) \right] \right\} \\ &= \frac{1}{|\Pi|} \sum_{\pi \in \Pi} \left\{ \sum_{d \in \pi, \hat{c}_d=1} \Delta(\hat{c}_d|\pi) \right. \\ &\quad \left. + o_\pi \left[\sum_{d \in \pi, \hat{c}_d=1} \frac{\Delta(c_d|\pi)}{\hat{P}(e_d=1)} - \sum_{d \in \pi, \hat{c}_d=1} \frac{\Delta(\hat{c}_d|\pi)}{\hat{P}(e_d=1)} \right] \right\}, \end{aligned} \quad (14)$$

where $\hat{\ell}_{IPW}(f, q|\pi) = \sum_{d \in \pi, \hat{c}_d=1} \frac{\Delta(\hat{c}_d|\pi)}{\hat{P}(e_d=1)}$ is the propensity score weighted imputed error, and $o_\pi = \mathbb{1}\{\pi = \pi_o\}$ indicates whether π is an observed ranked list. A nice property of the DR estimator is: if either the imputed error of any unobserved ranked list or the estimated propensities of any observed ranked list is accurate, the DR estimator is then unbiased [17, 55], which is recognized as *doubly robustness*.

Discussion. Careful readers may notice that our method is similar in spirit to result randomization. Ideally, if the pseudo-click labels generated from the user simulator are accurate for any unobserved ranked lists, learning from user simulators would be equivalent to learning from online result randomization, which is unbiased in principle [25, 46]. However, online result randomization is often impractical since it can negatively affect the user experience. Our

method overcomes the aforementioned limitations by leveraging the user simulator in a doubly robust way, and simultaneously obtains unbiased ranking models.

4.3 Bias and Variance of DR Estimator

In this section, we derive the bias and variance of MULTR, and prove its double robustness.

THEOREM 4.1. *Let $\delta_{\pi,d} = c_d\Delta(c_d|\pi) - \hat{c}_d\Delta(\hat{c}_d|\pi)$ be the error deviation, and $\rho_{q,d} = \frac{P(e_d=1) - \hat{P}(e_d=1)}{\hat{P}(e_d=1)}$ be the propensity deviation. The bias of the doubly robust (DR) estimator is*

$$\text{Bias}[\ell_{DR}(f, q|\pi_o)] = \left| \frac{1}{|\Pi|} \sum_{\pi \in \Pi} \sum_{d \in \pi} \rho_{\pi,d} \delta_{\pi,d} \right|. \quad (15)$$

As shown in Theorem 4.1, when the imputed error is accurate ($\delta_{q,d} = 0$) or the propensity estimation is accurate ($\rho_{q,d} = 0$), the bias of DR estimator $\text{Bias}[\ell_{DR}(f, q|\pi_o)] = 0$. Next, we derive the variance of the DR estimator.

THEOREM 4.2. *The variance of the DR estimator is*

$$\mathbb{V}_O[\ell_{DR}(f, q|\pi_o)] = \frac{1}{|\Pi|^2} \sum_{\pi \in \Pi} \sum_{d \in \pi} \left\{ \frac{P(e_d=1)(1-P(e_d=1))}{\hat{P}(e_d=1)^2} \cdot \delta_{q,d}^2 \right\} \quad (16)$$

In Theorem 4.2, we demonstrate that the variance of the DR estimator depends on the estimated propensity, *i.e.*, $\hat{P}(e_d=1)$, which may lead to a high variance problem. Recall the variance of the IPW estimator in Theorem 3.2; it is worth noting that our DR estimator can still reduce the variance of the IPW estimator, if any given observed ranked list satisfies $[c_d\Delta(c_d|\pi) - \hat{c}_d\Delta(\hat{c}_d|\pi)]^2 \leq [c_d\Delta(c_d|\pi)]^2$. Then, the variance of the DR estimator is smaller than that of the IPW-based estimator.

4.4 Learning Framework

We summarize the learning framework in Algorithm 1. It is worth noting that the proposed MULTR is a plug-in model, which can be seamlessly integrated into any IPW-based unbiased learning to rank framework without result randomization, such as DLA [2] and REM [47].

5 EXPERIMENTAL SETUP

To analyze the effectiveness of MULTR, we conduct two types of experiments. The first is simulation experiments based on two of the largest public learning-to-rank datasets. The second is an experiment based on the actual rank lists and user clicks collected from a commercial web search engine.

5.1 Simulation Experiment Setup

To fully investigate the spectrum of propensity estimation and performance of MULTR, we conduct experiments on two of the largest publicly available datasets:

¹Due to the large sample space of all possible ranked lists (*i.e.*, N documents have $N!$ permutations), we decrease the sample size in practice.

Algorithm 1: Model-based Unbiased Learning to Rank

Input : query set Q , an IPW-based framework F
Output: ranking model $f(\theta)$ and user simulator $g(\phi)$

- 1 Initialize the parameters θ, ϕ ;
- 2 **for** number of steps for **training the user simulator** **do**
- 3 Sample a batch of queries Q from Q , with observed ranked lists Π_o ;
- 4 Update ϕ , according to Eq. 12 ;
- 5 **end**
- 6 **for** number of steps for **training the unbiased ranking model** **do**
- 7 Sample a batch of queries Q from Q , with observed ranked lists Π_o ;
- 8 Obtain unobserved ranked lists Π_u by result randomization over Π_o ;
- 9 Generate pseudo-click labels $\hat{c}_d$ for all ranked list $\Pi = \Pi_o \cup \Pi_u$ by sampling on the click probabilities, according to Eq. 11¹ ;
- 10 Update θ , according to Eq. 14
- 11 **end**

- **Yahoo! LETOR**² comes from the Learn to Rank Challenge version 2.0 (Set 1), and it is one of the largest benchmark datasets widely used in unbiased learning to rank [2, 20]. It consists of 29,921 queries and 710K documents. Each query-document pair is represented by a 700-D feature vector, and annotated with 5-level relevance labels [4].
- **Istella-S**³ contains 33K queries and 3,408K documents (roughly 103 documents per query) sampled from a commercial Italian search engine. Each query-document pair is represented by 220 features and annotated with 5-level relevance judgments [28].

We follow the predefined data split of training, validation and testing of all datasets. The Yahoo! set splits the queries arbitrarily and uses 19,944 for training, 2,994 for validation and 6,983 for testing. The Istella-S dataset has been divided into train, validation and test sets according to a 60% – 20% – 20% scheme.

Click Simulation. We generate click data on both datasets with a two-step process as in Joachims et al. [25] and Ai et al. [2]. First, we train a Rank SVM model [22] using 1% of the training data with real relevance labels to generate the initial ranked list π_q for each query q . Then, we simulate the user browsing process and sample clicks from the initial ranked list by utilizing the simulation model. The simulation model generates clicks based on the examination hypothesis in Equation 5. Following the methodology proposed by Chapelle et al. [6], the relevance probability is set to be

$$\Pr(r_{d_i} = 1 | \pi_q) = \epsilon + (1 - \epsilon) \frac{2^y - 1}{2^{y_{\max}} - 1}, \quad (17)$$

where $y \in [0, y_{\max}]$ is the relevance label of the document d_i , and $y_{\max}$ is the maximum value of y , which is 4 on both datasets. ϵ is the noise level, which models click noise such that irrelevant documents

(i.e., $y = 0$) have a non-zero probability to be perceived as relevant and clicked. We fix $\epsilon = 0.1$ as the default setting.

We use the cascade model [11] to generate the examination probability. It is a context-aware click model, and a user is modeled as searching and clicking documents from top to bottom, and deciding whether to click each result before moving to the next; users stop examining a search result page after the first click. The cascade model is defined as,

$$\Pr(e_{d_i} = 1) = 1$$

$$\Pr(e_{d_j} | e_{d_{j-1}} = e, c_{d_{j-1}} = c) = e \cdot (1 - c) \quad (18)$$

where e and c are binary variables that indicate whether the document d is examined and clicked.

Baselines. To demonstrate the effectiveness of our proposed method, we compare with baseline methods, including IPW-based methods and bias modeling methods, which are widely used in ULTR problems.

- **DLA:** The Dual Learning Algorithm [2] treats the problem of unbiased learning to rank and unbiased propensity estimation as a dual problem, such that they can be optimized simultaneously.
- **REM:** The Regression EM model [47] uses an EM framework to estimate the propensity scores and ranking scores.
- **PairD:** The Pairwise Debiasing (PairD) Model [20] uses inverse propensity weighting for pairwise learning to rank.
- **PAL:** The Position-bias Aware Learning framework [16] is a bias modeling method. It introduces a position model to explicitly represent position bias, and jointly model the bias and relevance.
- **Oracle:** This model utilizes human experts' annotated labels to train the ranking model and test its performance. Its performance can be considered as an upper bound for the ranking model.
- **Naive:** This model just uses the raw click data to train the ranking model, without any correction. Its performance can be considered as a lower bound for the ranking model.

In addition, we also consider two variants of MULTR as follows,

- **Rand-MULTR:** We replace the well-trained user simulator with the model with random initialization, i.e., we skip lines 2-5 in the Algorithm 1. We refer to this variant as Rand-MULTR.
- **EIB-MULTR:** We alternate the doubly robust loss in Equation 14 with the error imputation based loss [18], defined as $\ell_{EIB} = \frac{1}{|\Pi|} \sum_{\pi \in \Pi} \ell_{naive}(f, q | \pi)$. It is called EIB since it uses the user simulator to compute an imputed error, i.e., the estimated values of the ranking loss, for each unobserved ranked list, and then used it to estimate the true ranking loss for all the ranked lists. We refer this variant as EIB-MULTR.

Experimental Protocols. We implement MULTR and use the baselines in ULTRA⁴ [42] to conduct our experiments. In particular, MULTR is integrated with DLA, as we consider it to be the best method to estimate propensity. For each query, only the top $N = 10$ documents are assumed to be displayed to the users. For both datasets, all models are trained with synthetic clicks. Following the setting in Ai et al. [2], the click sessions for training are generated on the fly. We fix the batch size to 256 and train each model for 10K steps. The user simulator is also trained with batch size of 256 and 10K steps; afterwards, we fix it to generate pseudo-click

²<https://webscope.sandbox.yahoo.com/>

³<http://quickrank.isti.cnr.it/istella-dataset/>

⁴https://github.com/ULTR-Community/ULTRA_pytorch/

labels. We use the AdaGrad optimizer [14] and tune learning rates from 0.01 to 0.05 for each unbiased learning-to-rank algorithm.

In our experiments, we train neural networks for our ranking functions. All reported results are produced using a model with three hidden layers with size [512, 256, 128] respectively, with the ELU [10] activation function and 0.1 dropout [40]. In terms of imputation model, the dimension of the hidden states is 64. We tune the L_2 regularization coefficient λ , μ in range of $\{1e^{-5}, 1e^{-4}, \dots, 1\}$, and the number of pseudo samples in $\{2, 4, 8, 16, 24, 32\}$ based on results in the validation set.

To evaluate all methods, we use the normalized Discounted Cumulative Gain (nDCG) [21] and the Expected Reciprocal Rank (ERR) [6]. For both metrics, we report the results at rank 1, 3, 5, and 10 to show the performance of models at different positions. Following Ai *et al.* [2], statistical differences are computed based on the Fisher randomization test [39] with $p \leq 0.05$.

5.2 Real Click Experiment Setup

In order to show the effectiveness of MULTR in practice, we also conduct experiments on click data **Tiangong-ULTR**⁵ collected from a commercial web search engine [2, 3]. It contains 3,449 queries written by real search engine users and the corresponding top 10 results are sampled from a two-week search log collected on Sogou⁶. The raw HTML documents are downloaded, and then the rank lists that cannot be crawled are removed. After cleaning, there are 333,813 documents, 71,106 ranked lists and 3,268,177 anonymous click sessions.

Feature Extraction. To train the learning-to-rank models, features are extracted based on the text of queries and documents. The ranking features are constructed based on the URL, title, content and whole text of the documents and queries [2]. In total, each query-document pair has 33 features.

Evaluation. Tiangong-ULTR provides a test set with 100 queries written by real users. Each query has 100 candidate documents (retrieved by BM25) and each query-document pair has 5-level relevance annotation. We train the ranking model in MULTR and base-lines on the training set with clicks following the same protocols in the simulation experiments, and we evaluate their performance on the test set with human annotations. Similar to the simulation experiments, we report nDCG and ERR for all models.

6 RESULTS AND ANALYSIS

In this section, we discuss the results of our proposed model-based unbiased learning to rank method with existing approaches using both simulated and real-world experiments. In general, we expect the experimental results to answer the following research questions:

- **RQ1:** Can MULTR outperform state-of-the-art unbiased learning to rank methods?
- **RQ2:** What influence do variant designs have on MULTR?
- **RQ3:** How does the sample number of unobserved ranked lists influence the performance of MULTR?
- **RQ4:** How does MULTR perform on real click logs?

⁵<http://www.thuir.cn/data-tiangong-ultr/>

⁶<https://www.sogou.com/>

6.1 Performance Comparison (RQ1)

To answer RQ1, we compare MULTR with other unbiased learning-to-rank algorithms in the simulation experiments. Table 1 summarizes the performance of different unbiased learning to rank algorithms under various click situations. We find that:

- Our model-based unbiased learning to rank achieves the best performance among all the state-of-the-art methods in terms of nDCG and ERR, which indicates our model is robust when propensity cannot be accurately estimated.
- Reducing variance can enhance performance on unbiased learning to rank. As MULTR shares the same method with DLA in estimating propensity, the improvements over DLA demonstrate the necessity of handling variance in IPW-based methods.
- The naive method may achieve better performance than some unbiased learning to rank methods when clicks are generated by a cascade model. One explanation is that documents displayed at a lower rank have a higher opportunity to be examined; thus they receive more clicks, which alleviates the bias in click data. This observation confirms that a mismatch between propensity estimation and real bias could lead to performance even worse than raw click data.
- The oracle model with human annotation consistently achieves the best performance. It implies that there is still room to improve in unbiased learning to rank methods.

6.2 Ablation Study (RQ2)

MULTR has specific design features, including a context-aware user simulator and a doubly robust loss function to train rankers. To demonstrate their effectiveness, we analyze their respective impacts on the model's performance via ablation study. The experimental results of its two variants on two datasets are summarized in Table 1. We analyze their respective effects as follows.

(1) Rand-MULTR: When we replace the well-trained user simulator with one from random initialization, the performance of Rand-MULTR significantly degrades. This observation verifies the necessity of an accurate user simulator for the doubly robust estimator learning process: when the user simulator has a high bias due to inaccurate click imputations, it will mislead the rankers.

(2) EIB-MULTR: When we use the same treatments for real clicks from observations and pseudo clicks from user simulators, the performance of EIB-MULTR suffers a significant decrease. This suggests, naturally, that there is a discrepancy between real clicks and pseudo clicks. The doubly robust approach effectively leverages pseudo clicks and improves the performance of rankers.

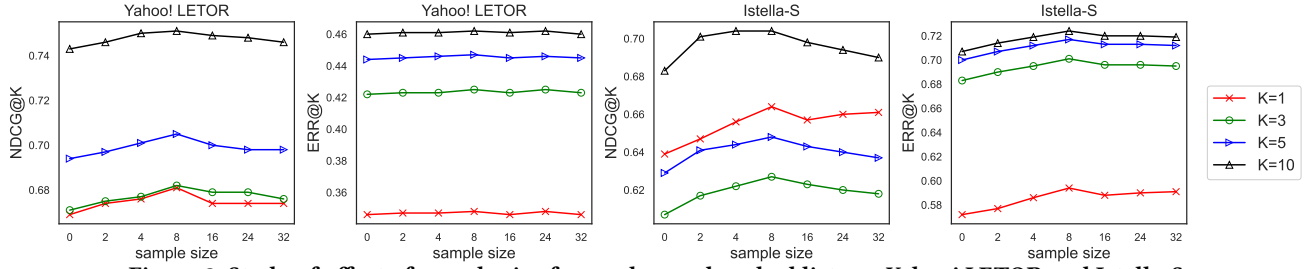
6.3 Parameter Sensitivity Study (RQ3)

As we have explained previously, when N is the maximum number of documents that can be shown to users, there will be $N!$ candidate ranked lists, which could be large. Therefore, we decrease the number of samples for unobserved ranked lists in practice. To investigate its impact in MULTR, we vary the sample size for unobserved ranked lists in the range of $\{0, 2, 4, 8, 16, 24, 32\}$. Figure 3 shows the nDCG@K and ERR@K for MULTR with respect to different sample sizes on both datasets.

First, MULTR with sample number 0, *i.e.*, we merely sample from unobserved candidate rank lists, derives the worst performance.

Table 1: A comparison of the overall performance MULTR and competing methods on Yahoo! and Istella-S datasets. “*” indicates a statistically significant improvement over the best baseline.

Methods	Yahoo! LETOR								Istella-S							
	NDCG@K				ERR@K				NDCG@K				ERR@K			
	K = 1	K = 3	K = 5	K = 10	K = 1	K = 3	K = 5	K = 10	K = 1	K = 3	K = 5	K = 10	K = 1	K = 3	K = 5	K = 10
Oracle	0.689	0.691	0.714	0.761	0.350	0.428	0.449	0.465	0.671	0.641	0.667	0.729	0.598	0.709	0.725	0.731
MULTR	0.681*	0.682*	0.705*	0.751*	0.348*	0.426*	0.447*	0.463*	0.664*	0.627*	0.648*	0.704*	0.594*	0.701*	0.717*	0.724*
DLA	0.669	0.671	0.694	0.743	0.346	0.422	0.444	0.460	0.639	0.607	0.629	0.683	0.572	0.683	0.700	0.707
REM	0.636	0.638	0.661	0.711	0.336	0.410	0.432	0.448	0.590	0.551	0.568	0.618	0.528	0.640	0.659	0.669
PairD	0.653	0.662	0.687	0.738	0.333	0.413	0.436	0.451	0.604	0.586	0.617	0.687	0.537	0.661	0.680	0.688
PAL	0.645	0.660	0.683	0.735	0.328	0.410	0.432	0.448	0.620	0.595	0.623	0.691	0.553	0.672	0.690	0.698
Naive	0.651	0.660	0.685	0.737	0.332	0.411	0.434	0.450	0.613	0.592	0.622	0.690	0.546	0.667	0.686	0.694
Rand-MULTR	0.667	0.670	0.693	0.743	0.341	0.419	0.441	0.457	0.633	0.604	0.631	0.693	0.564	0.681	0.698	0.706
EIB-MULTR	0.674	0.678	0.700	0.749	0.346	0.423	0.445	0.460	0.653	0.611	0.631	0.684	0.585	0.690	0.707	0.714

**Figure 3: Study of effect of sample size for unobserved ranked lists on Yahoo! LETOR and Istella-S.**

This observation shows that the well-trained user simulator enables the unobserved rank lists to provide rankers with useful information. Second, when the sampling number increases from 0 to 8, the performance at all levels increases. This shows that sampling more unobserved rank lists can bring more beneficial information to rankers. Third, overly large sampling does not show a significant improvement, even though we should sample all unobserved rank lists in theory. We even observe that NDCG starts to decrease when the sample number increases from 8 to 32. One possible reason could be that the observed rank lists are typically sparse in practice, meaning that we cannot ensure that the rankers obtain sufficient information. For both datasets, the optimal sampling size is 8. In conclusion, setting the sampling size too conservatively or too aggressively may adversely affect the ranking performance, and too aggressively also brings in unnecessary computational cost.

6.4 Real Data Study (RQ4)

As we have demonstrated that the proposed MULTR works well in the simulation study, we would further investigate how it performs on real click logs recorded by the search engine. We compare MULTR with existing unbiased learning to rank algorithms on real data. The experimental results are summarized in Table 2. From the results, we can see that MULTR outperforms state-of-the-art unbiased learning to rank algorithms. This shows that the user simulator in MULTR provides reliable pseudo-clicks for the ranker, which enhances the ranking performance. In addition, propensity models in both MULTR and baseline methods are developed upon position-based examination assumption, which makes accurate propensity estimation infeasible on real data. Therefore, they are more likely to suffer from the high variance problem. The superior performance

of MULTR over baseline methods verifies the necessity of reducing the variance via doubly-robust learning.

Table 2: Comparison of MULTR and competing methods on Tiangong-ULTR.

Methods	NDCG@K				ERR@K			
	K = 1	K = 3	K = 5	K = 10	K = 1	K = 3	K = 5	K = 10
MULTR	0.483	0.465	0.469	0.476	0.448	0.578	0.606	0.618
DLA	0.472	0.443	0.425	0.443	0.443	0.571	0.594	0.610
REM	0.450	0.455	0.457	0.476	0.422	0.566	0.594	0.606
PairD	0.388	0.371	0.369	0.383	0.364	0.494	0.522	0.542
PAL	0.365	0.384	0.395	0.413	0.342	0.483	0.516	0.533
Naive	0.412	0.413	0.408	0.411	0.386	0.528	0.556	0.570

7 CONCLUSION

In this work, we propose MULTR, a model-based unbiased learning to rank framework. We first design a context-aware user simulator to produce labels for unobserved ranked lists as supervision signals, which addressed data sparsity for long-tail queries. However, the natural discrepancy between pseudo and actual clicks could mislead the ranking models, as pseudo data cannot be as accurate as real users. Therefore, we resort to DR approaches. In particular, we take the observation of the ranked lists as the treatment, which addresses the treatment unobservation when naively applying existing DR methods. Afterward, we take the pseudo-click data as data imputation, and propose a doubly-robust learning algorithm to obtain unbiased ranking models. Theoretical analysis reveals that our method is more robust than existing methods. Extensive experiments on benchmark datasets, including simulated datasets and real click logs, demonstrate that the proposed model-based method consistently outperforms state-of-the-art methods.

REFERENCES

- [1] Qingyao Ai, Keping Bi, Jiafeng Guo, and W. Bruce Croft. 2018. Learning a Deep Listwise Context Model for Ranking Refinement. In *SIGIR 2018*.
- [2] Qingyao Ai, Keping Bi, Cheng Luo, Jiafeng Guo, and W. Bruce Croft. 2018. Unbiased Learning to Rank with Unbiased Propensity Estimation. In *SIGIR 2018*.
- [3] Qingyao Ai, Jiaxin Mao, Yiqun Liu, and W. Bruce Croft. [n. d.]. Unbiased Learning to Rank: Theory and Practice. In *CIKM 2018*.
- [4] Olivier Chapelle and Yi Chang. 2011. Yahoo! Learning to Rank Challenge Overview. In *Proceedings of the Yahoo! Learning to Rank Challenge, held at ICML 2010, Haifa, Israel, June 25, 2010 (JMLR Proceedings)*.
- [5] Olivier Chapelle, Yi Chang, and Tie-Yan Liu (Eds.). 2011. *Proceedings of the Yahoo! Learning to Rank Challenge, held at ICML 2010, Haifa, Israel, June 25, 2010*. JMLR Proceedings, Vol. 14. JMLR.org.
- [6] Olivier Chapelle, Donald Metzler, Ya Zhang, and Pierre Grinspan. 2009. Expected reciprocal rank for graded relevance. In *CIKM 2009*.
- [7] Olivier Chapelle and Ya Zhang. 2009. A dynamic bayesian network click model for web search ranking. In *WWW 2009*.
- [8] Xiaokai Chu, Jiahu Zhao, Lixin Zou, and Dawei Yin. 2022. H-ERNIE: A Multi-Granularity Pre-Trained Language Model for Web Search. In *CIKM '22*.
- [9] Aleksandr Chuklin, Ilya Markov, and Maarten de Rijke. 2016. Click Models for Web Search and their Applications to IR: WSDM 2016 Tutorial. In *WSDM 2016*.
- [10] Djork-Arné Clevert, Thomas Unterthiner, and Sepp Hochreiter. 2016. Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs). In *ICLR 2016*.
- [11] Nick Craswell, Onno Zoeter, Michael J. Taylor, and Bill Ramsey. 2008. An experimental comparison of click position-bias models. In *WSDM 2008*.
- [12] Xinyi Dai, Jiawei Hou, Qing Liu, Yunxia Xi, Ruiming Tang, Weinan Zhang, Xiquang He, Jun Wang, and Yong Yu. 2020. U-rank: Utility-oriented Learning to Rank with Implicit Feedback. In *CIKM '20*.
- [13] Mostafa Dehghani, Hamed Zamani, Aliaksei Severyn, Jaap Kamps, and W. Bruce Croft. 2017. Neural Ranking Models with Weak Supervision. In *SIGIR 2017*.
- [14] John C. Duchi, Elad Hazan, and Yoram Singer. 2011. Adaptive Subgradient Methods for Online Learning and Stochastic Optimization. *J. Mach. Learn. Res.* 12 (2011), 2121–2159.
- [15] Georges Dupret and Benjamin Piwowarski. 2008. A user browsing model to predict search engine click data from past observations. In *SIGIR 2008*.
- [16] Huifeng Guo, Jinkai Yu, Qing Liu, Ruiming Tang, and Yuzhou Zhang. 2019. PAL: a position-bias aware learning framework for CTR prediction in live recommender systems. In *RecSys 2019*.
- [17] Siyuan Guo, Lixin Zou, Yiding Liu, Wenwen Ye, Suqi Cheng, Shuaiqiang Wang, Hechang Chen, Dawei Yin, and Yi Chang. 2021. Enhanced Doubly Robust Learning for Debiasing Post-Click Conversion Rate Estimation. In *SIGIR 2021*.
- [18] José Miguel Hernández-Lobato, Neil Houlsby, and Zoubin Ghahramani. 2014. Probabilistic Matrix Factorization with Non-random Missing Data. In *ICML 2014 (JMLR Workshop and Conference Proceedings)*.
- [19] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long Short-Term Memory. *Neural Comput.* 9, 8 (1997), 1735–1780.
- [20] Ziniu Hu, Yang Wang, Qu Peng, and Hang Li. 2019. Unbiased LambdaMART: An Unbiased Pairwise Learning-to-Rank Algorithm. In *WWW 2019*.
- [21] Kalervo Järvelin and Jaana Kekäläinen. 2002. Cumulated gain-based evaluation of IR techniques. *ACM Trans. Inf. Syst.* 20, 4 (2002), 422–446.
- [22] Thorsten Joachims. 2006. Training linear SVMs in linear time. In *SIGKDD 2006*.
- [23] Thorsten Joachims, Laura A. Granka, Bing Pan, Helene Hembrooke, and Geri Gay. 2005. Accurately interpreting clickthrough data as implicit feedback. In *SIGIR 2005*.
- [24] Thorsten Joachims, Laura A. Granka, Bing Pan, Helene Hembrooke, Filip Radlinski, and Geri Gay. 2007. Evaluating the accuracy of implicit feedback from clicks and query reformulations in Web search. *ACM Trans. Inf. Syst.* 25, 2 (2007), 7.
- [25] Thorsten Joachims, Adith Swaminathan, and Tobias Schnabel. 2017. Unbiased Learning-to-Rank with Biased Feedback. In *WSDM 2017*.
- [26] Haruka Kiyohara, Yuta Saito, Tatsuya Matsui, Yusuke Narita, Nobuyuki Shimizu, and Yasuo Yamamoto. 2022. Doubly Robust Off-Policy Evaluation for Ranking Policies under the Cascade Behavior Model. In *WSDM '22*.
- [27] Dugang Liu, Pengxiang Cheng, Zhenhua Dong, Xiuqiang He, Weiye Pan, and Zhong Ming. 2020. A General Knowledge Distillation Framework for Counterfactual Recommendation via Uniform Data. In *SIGIR 2020*.
- [28] Claudio Lucchese, Franco Maria Nardini, Salvatore Orlando, Raffaele Perego, Fabrizio Silvestri, and Salvatore Trani. 2016. Post-Learning Optimization of Tree Ensembles for Efficient Ranking. In *SIGIR 2016*.
- [29] Jiaxin Mao, Zhumin Chu, Yiqun Liu, Min Zhang, and Shaoping Ma. 2019. Investigating the Reliability of Click Models. In *ICTIR 2019*.
- [30] Jiaxin Mao, Cheng Luo, Min Zhang, and Shaoping Ma. 2018. Constructing Click Models for Mobile Search. In *SIGIR 2018*.
- [31] Harrie Oosterhuis. 2022. Doubly-Robust Estimation for Unbiased Learning-to-Rank from Position-Biased Click Feedback. *arXiv preprint arXiv:2203.17118* (2022).
- [32] Zohreh Ovaisi, Ragib Ahsan, Yifan Zhang, Kathryn Vasilaky, and Elena Zheleva. 2020. Correcting for Selection Bias in Learning-to-rank Systems. In *WWW 2020*.
- [33] Matthew Richardson, Ewa Dominowska, and Robert Ragno. 2007. Predicting clicks: estimating the click-through rate for new ads. In *WWW 2007*.
- [34] Paul R Rosenbaum and Donald B Rubin. 1983. The central role of the propensity score in observational studies for causal effects. *Biometrika* 70, 1 (1983).
- [35] Yuta Saito. 2020. Asymmetric Tri-training for Debiasing Missing-Not-At-Random Explicit Feedback. In *SIGIR 2020*.
- [36] Yuta Saito. 2020. Doubly Robust Estimator for Ranking Metrics with Post-Click Conversions. In *RecSys 2020*.
- [37] Mark Sanderson. 2010. Test Collection Based Evaluation of Information Retrieval Systems. *Found. Trends Inf. Retr.* 4, 4 (2010), 247–375.
- [38] Chengyao Shen and Qi Zhao. 2014. Webpage Saliency. In *ECCV 2014*.
- [39] Mark D. Smucker, James Allan, and Ben Carterette. 2007. A comparison of statistical significance tests for information retrieval evaluation. In *CIKM 2007*.
- [40] Nitish Srivastava, Geoffrey E. Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. 2014. Dropout: a simple way to prevent neural networks from overfitting. *J. Mach. Learn. Res.* 15, 1 (2014), 1929–1958.
- [41] Adith Swaminathan and Thorsten Joachims. 2015. The Self-Normalized Estimator for Counterfactual Learning. In *NeurIPS 2015*.
- [42] Anh Tran, Tao Yang, and Qingyao Ai. 2021. ULTRA: An Unbiased Learning To Rank Algorithm Toolbox. In *CIKM 2021*.
- [43] Ali Vardasbi, Maarten de Rijke, and Ilya Markov. 2020. Cascade Model-based Propensity Estimation for Counterfactual Learning to Rank. In *SIGIR 2020*.
- [44] Ali Vardasbi, Harrie Oosterhuis, and Maarten de Rijke. 2020. When Inverse Propensity Scoring does not Work: Affine Corrections for Unbiased Learning to Rank. In *CIKM '20*.
- [45] Hongning Wang, ChengXiang Zhai, Anlei Dong, and Yi Chang. 2013. Content-aware click modeling. In *WWW 2013*.
- [46] Xuanhui Wang, Michael Bendersky, Donald Metzler, and Marc Najork. 2016. Learning to Rank with Selection Bias in Personal Search. In *SIGIR 2016*.
- [47] Xuanhui Wang, Nadav Golbandi, Michael Bendersky, Donald Metzler, and Marc Najork. 2018. Position Bias Estimation for Unbiased Learning to Rank in Personal Search. In *WSDM 2018*.
- [48] Wenwen Ye, Yiding Liu, Lixin Zou, Hengyi Cai, Suqi Cheng, Shuaiqiang Wang, and Dawei Yin. 2022. Fast semantic matching via flexible contextualized interaction. In *WSDM '22*.
- [49] Bo-Wen Yuan, Jui-Yang Hsia, Meng-Yuan Yang, Hong Zhu, Chih-Yao Chang, Zhenhua Dong, and Chih-Jen Lin. 2019. Improving Ad Click Prediction by Considering Non-displayed Events. In *CIKM 2019*.
- [50] Yisong Yue, Rajan Patel, and Hein Roehrig. 2010. Beyond position bias: examining result attractiveness as a source of presentation bias in clickthrough data. In *WWW 2010*.
- [51] Hamed Zamani, Bhaskar Mitra, Xia Song, Nick Craswell, and Saurabh Tiwary. 2018. Neural Ranking Models with Multiple Document Fields. In *WSDM 2018*.
- [52] Junqi Zhang, Yiqun Liu, Jiaxin Mao, Weizhi Ma, Jiazheng Xu, Shaoping Ma, and Qi Tian. 2022. User Behavior Simulation for Search Result Re-Ranking. *ACM Transactions on Information Systems* (2022).
- [53] Junqi Zhang, Jiaxin Mao, Yiqun Liu, Ruizhe Zhang, Min Zhang, Shaoping Ma, Jun Xu, and Qi Tian. 2019. Context-Aware Ranking by Constructing a Virtual Environment for Reinforcement Learning. In *CIKM 2019*.
- [54] Honglei Zhuang, Zhen Qin, Xuanhui Wang, Michael Bendersky, Xinyu Qian, Po Hu, and Dan Chary Chen. 2021. Cross-Positional Attention for Debiasing Clicks. In *WWW 2021*.
- [55] Lixin Zou, Changying Hao, Hengyi Cai, Shuaiqiang Wang, Suqi Cheng, Zhicong Cheng, Wenwen Ye, Simiu Gu, and Dawei Yin. 2022. Approximated Doubly Robust Search Relevance Estimation. In *CIKM '22*.
- [56] Lixin Zou, Weixue Lu, Yiding Liu, Hengyi Cai, Xiaokai Chu, Dehong Ma, Daiting Shi, Yu Sun, Zhicong Cheng, Simiu Gu, et al. 2022. Pre-trained Language Model based Retrieval and Ranking for Web Search. *ACM Transactions on the Web* (2022).
- [57] Lixin Zou, Haitao Mao, Xiaokai Chu, Jiliang Tang, Wenwen Ye, Shuaiqiang Wang, and Dawei Yin. 2022. A Large Scale Search Dataset for Unbiased Learning to Rank. *arXiv preprint arXiv:2207.03051* (2022).
- [58] Lixin Zou, Shengqiang Zhang, Hengyi Cai, Dehong Ma, Suqi Cheng, Shuaiqiang Wang, Daiting Shi, Zhicong Cheng, and Dawei Yin. 2021. Pre-trained language model based ranking in Baidu search. In *KDD '21*.